

Multithreaded Linux Application Using POSIX Threads and Mutexes

LINUX SYSTEM MONITORING APPLICATION

TEAM MEMBERS :

2500031613 - U.Moksha

2520030408 - Jamuna

2520030416 - Keerthi

SECTION : 7

TEAM NO. : 5

FACULTY NAME :

K.HEMA

Introduction

- Linux provides several system resources that need to be monitored continuously, such as CPU, memory, disk, network activity, and running processes.
- The Linux System Monitor is a C-based multithreaded application developed for Linux systems.
- The application uses POSIX threads to monitor different system resources concurrently.
- It also uses mutex synchronization to safely access shared data between multiple threads

Problem Statement

Monitoring multiple Linux system resources simultaneously requires concurrent execution and safe data sharing.

This project uses POSIX threads and mutex synchronization to monitor system resources efficiently while preventing race conditions.

Objectives

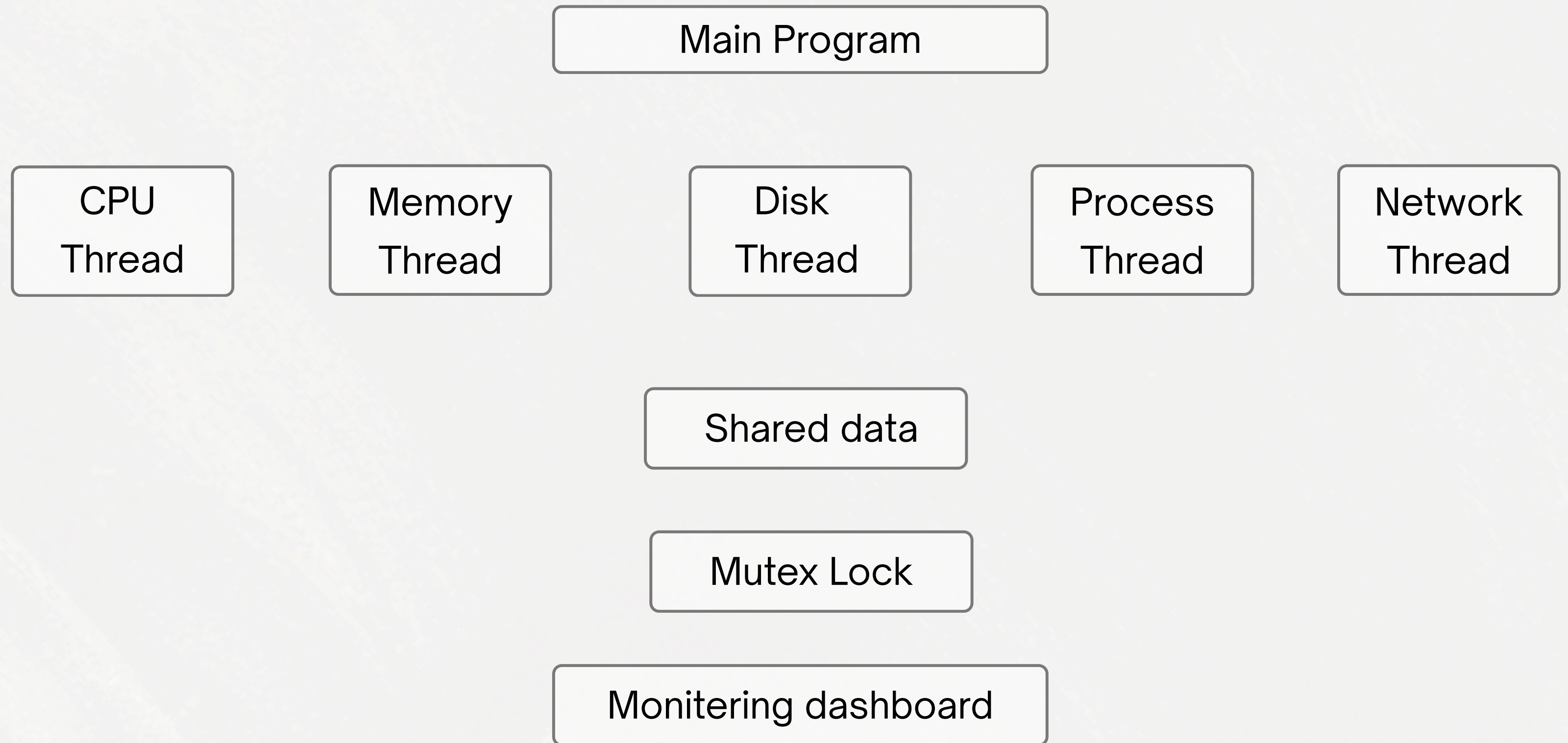
The main objectives of this project are:

- To develop a Linux-based system monitoring application using C.
- To implement multiple POSIX threads.
- To monitor CPU, memory, disk, network, and processes concurrently.
- To understand the concept of shared resources.
- To implement mutex synchronization.
- To demonstrate a race condition.
- To show how mutexes prevent incorrect access to shared data.
- To display system information in a real-time monitoring dashboard.

Technologies Used

- C
- Linux
- POSIX Threads
- Mutex
- /proc filesystem
- GCC
- Makefile

System Architecture



Multithreading & Mutex

- 5 monitoring threads run concurrently.
- Each thread performs a specific monitoring task.
- Shared data is protected using **pthread_mutex_lock()** and **pthread_mutex_unlock()**.
- Critical sections prevent simultaneous modification.

Advantages

- Provides real-time system information.
- Uses concurrent execution through POSIX threads.
- Separates monitoring tasks into independent modules.
- Demonstrates synchronization using mutexes.
- Demonstrates the effect of race conditions.
- Uses Linux's /proc filesystem for system information.
- Provides a simple and readable monitoring dashboard.
- Uses modular source files for easier maintenance.

Conclusion

The project implements:

- POSIX threads for concurrent execution.
- Shared resources between multiple threads.
- Mutex synchronization for safe access to shared data.
- Critical sections for protected operations.
- Race-condition demonstration to show synchronization problems.
- CPU, memory, disk, network, and process monitoring.
- A continuously updating system monitoring dashboard.

The project demonstrates how operating-system concepts such as threads, concurrency, synchronization, and mutual exclusion can be applied to a real Linux application.

Thank You